CSCI 5540 - Fall 26 Tutorial 7 Jia Chaobo

verl: the system around an RL training loop

Coordinate generation, feedback and optimization across distributed engines.

RL workflow

Express PPO / GRPO data dependencies and choose a trainer mode.

Execution engines

Reuse training and inference backends through modular interfaces.

System services

Move trajectories, synchronize weights and allocate GPU resources.

Source snapshot: main@3c6d9b4 | Latest published release: v0.9.1 | 26 October 2026

Learning path: RL loop → architecture → components → execution modes → practical use.

RL training: generate, score, update

1 Generate
Policy produces token sequences

→

2 Evaluate
Rewards, log probabilities, values

→

3 Optimize
Advantages drive policy updates

Role	What it provides	When needed
Actor / rollout	Trainable policy / sampled trajectories	Every policy-learning run
Reference	A fixed policy for KL regularization	When a KL objective uses it
Critic	A learned estimate of future return	Typical PPO with GAE
Reward	Feedback from rules, models or tools	A task-specific training signal

GRPO replaces the learned value baseline with group-relative feedback; the overall loop remains.

verl architecture: trainer, engines and services

1 Model engine

Forward / backward, optimizer and sharding.

2 Rollout engine

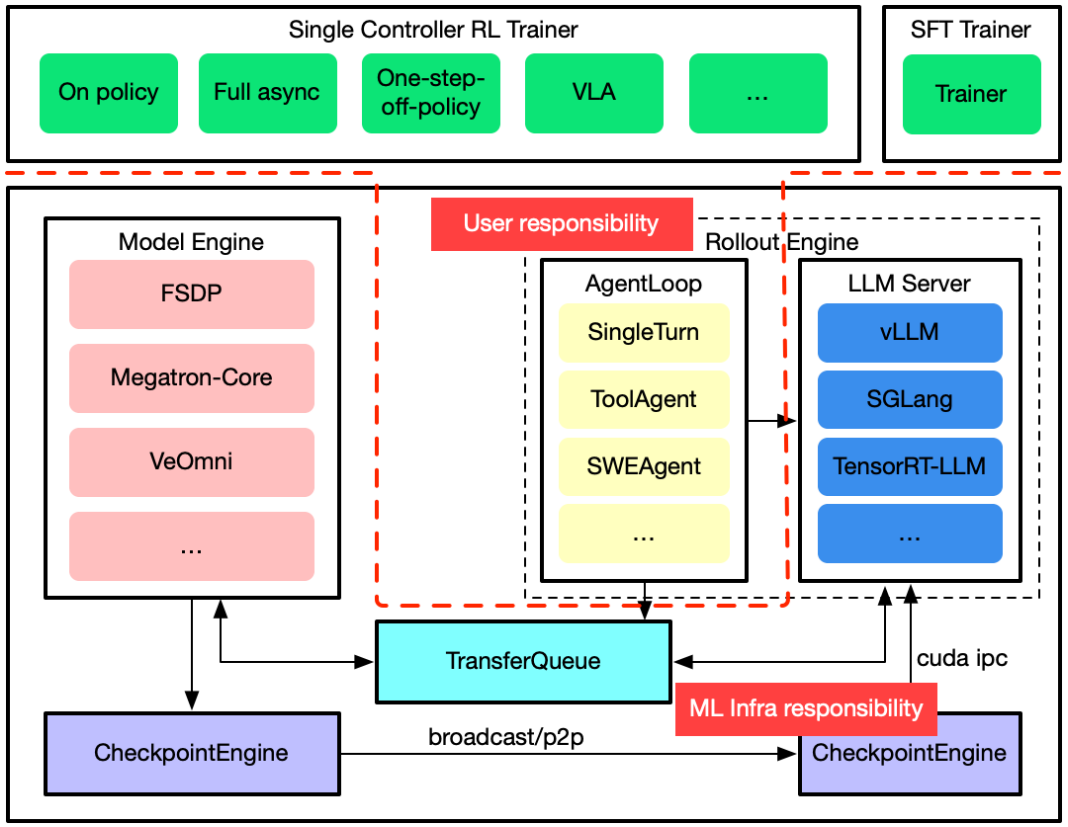
LLM servers plus task-specific agent loops.

3 TransferQueue

Trajectory fields and readiness metadata.

4 Checkpoint engine

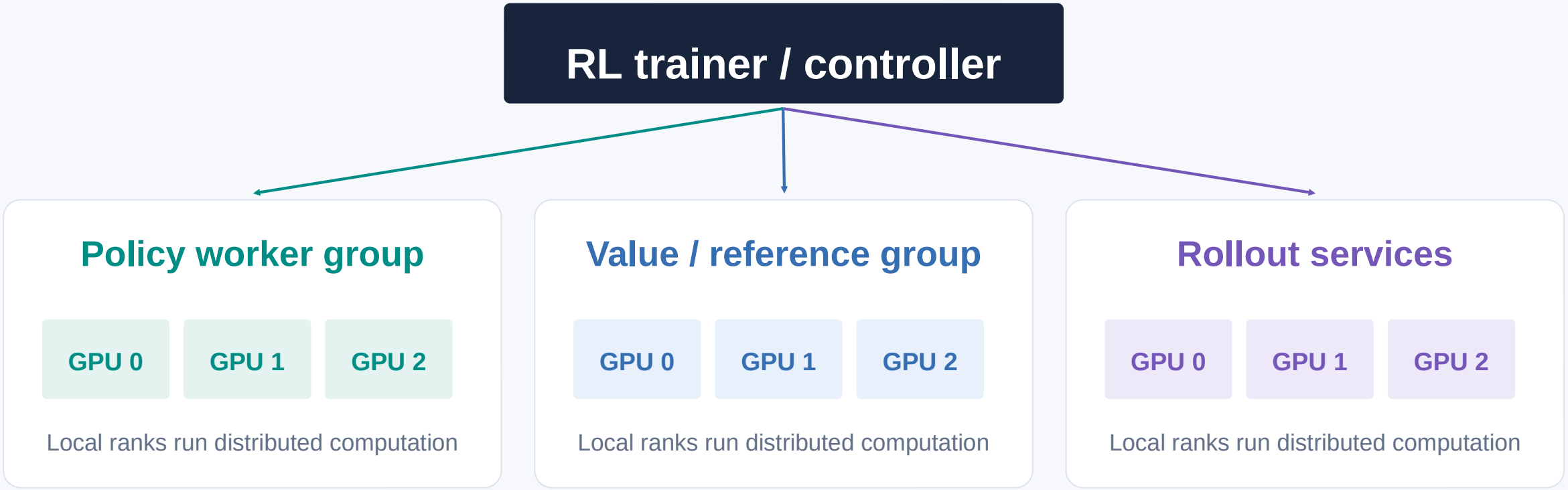
Updated weights from trainer to rollout.



A trainer builds an RL pipeline by composing these components; backends implement the heavy work.

Official architecture overview | Figure linked from the current official README

A controller coordinates distributed worker groups



The controller schedules tasks; engine backends manage collectives and parallel ranks.

WorkerGroup handles remote invocation and dispatch; GPU kernels remain inside the execution engines.

Follow one run from configuration to the V1 training loop

CURRENT ENTRY PATH

```
python -m verl.trainer.main_ppo

Hydra config → TaskRunnerV1
get_trainer_cls(trainer_mode)
trainer.init()
AgentLoopManagerTQ.create(...)
trainer.fit(agent_loop_manager)
```

1 **Configure**

Choose algorithm, model, data, engines and GPU layout.

2 **Initialize**

Create workers, queues, rollout replicas and reward services.

3 **Train**

The selected V1 mode controls sampling and weight-sync timing.

Read trainer_base.py for the shared learning step; mode subclasses specialize lifecycle hooks.

Workers compose roles; engines run the models

ActorRolloutRefWorker

Subcomponents are created from the configured role.

actor	TrainingWorker → trainable model engine
ref	TrainingWorker → forward-only model engine
rollout	BaseRollout + checkpoint-engine integration

TrainingWorker

One BaseEngine
+ optimizer configuration
+ profiler / checkpoint APIs

Also used for standalone roles, including the value critic.

Role composition and backend selection are independent; one worker API can wrap different engines.

The model engine hides backend-specific training details

Backend family	Main execution idea
FSDP / FSDP2	Shard model state across data-parallel ranks
Megatron	Combine data, tensor, pipeline, context and expert parallelism
VeOmni	FSDP-based training with sequence / expert parallelism
TorchTitan / Automodel	Additional model-engine integrations

COMMON RESPONSIBILITIES

- Initialize model and optimizer
- Run train / inference batches
- Load or offload model state
- Export parameters for rollout
- Save and restore training state

The selected engine must support your model, hardware and parallelism.

Two design choices: parallelism and GPU placement

DP
Different data
Model replicas

TP
Split tensor operations

PP
Split layers into pipeline stages

CP / SP
Split sequence computation

EP
Distribute MoE experts

Colocated pool

Training and rollout reuse GPUs at different times.

GPU GPU GPU GPU GPU

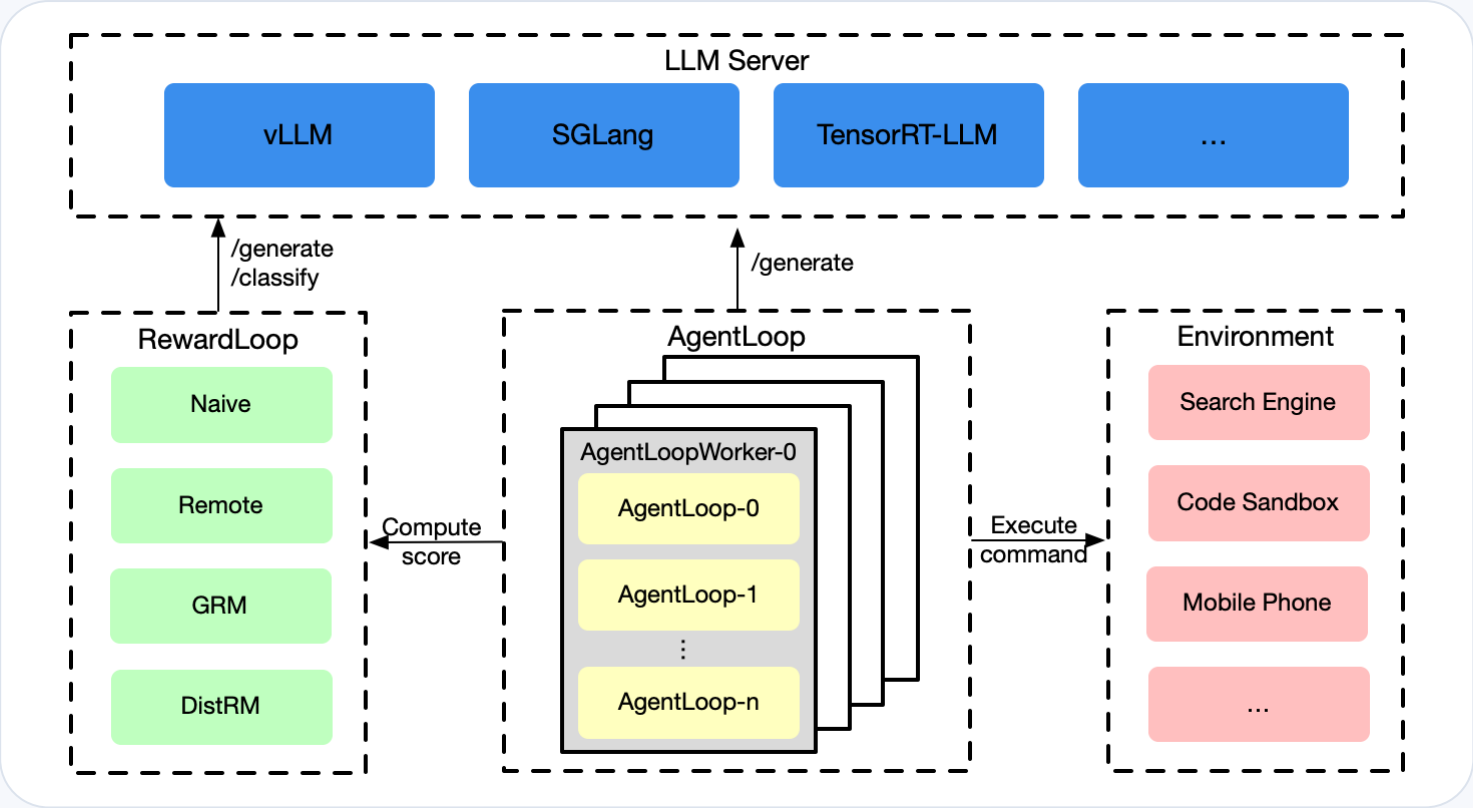
Separate pools

Rollout and training can run on independent GPU pools.

GPU GPU GPU GPU GPU

Choose parallel groups for each engine, then choose how roles share or separate physical resources.

Rollout is a server-based request pipeline



1 Request

An AgentLoop asks an LLM server to generate tokens.

2 Serve

Inference backends batch active requests and decode.

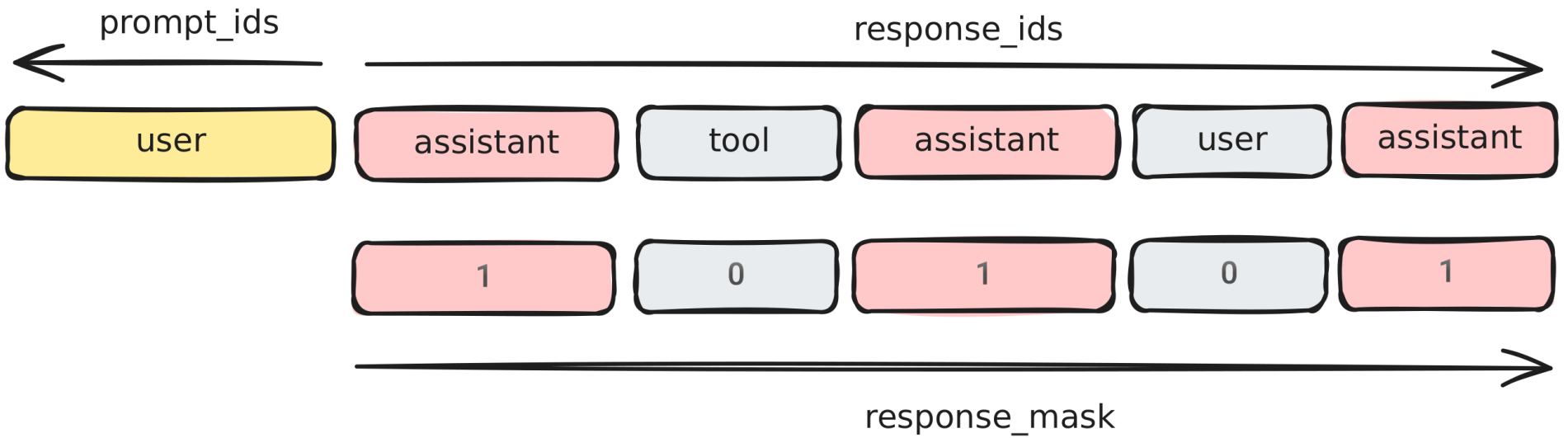
3 Continue

Return tokens, call tools, or finish and compute reward.

Asynchronous server requests support concurrent trajectories, multi-turn tasks and dynamic batching.

Rollout engine design | Official documentation figure; exact backend support is version-specific

Agent trajectories: tokens, masks and log probabilities



AgentLoopBase.run()

Returns prompt_ids, response_ids and aligned metadata.

response_mask

1 = generated token; 0 = observation or padding token.

Use token-in / token-out trajectories so optimization targets the tokens the policy actually generated.

Rewards and advantages are separate choices

Rule-based

Answer checks or executable tests

Model-based

Discriminative or generative judges

Combined

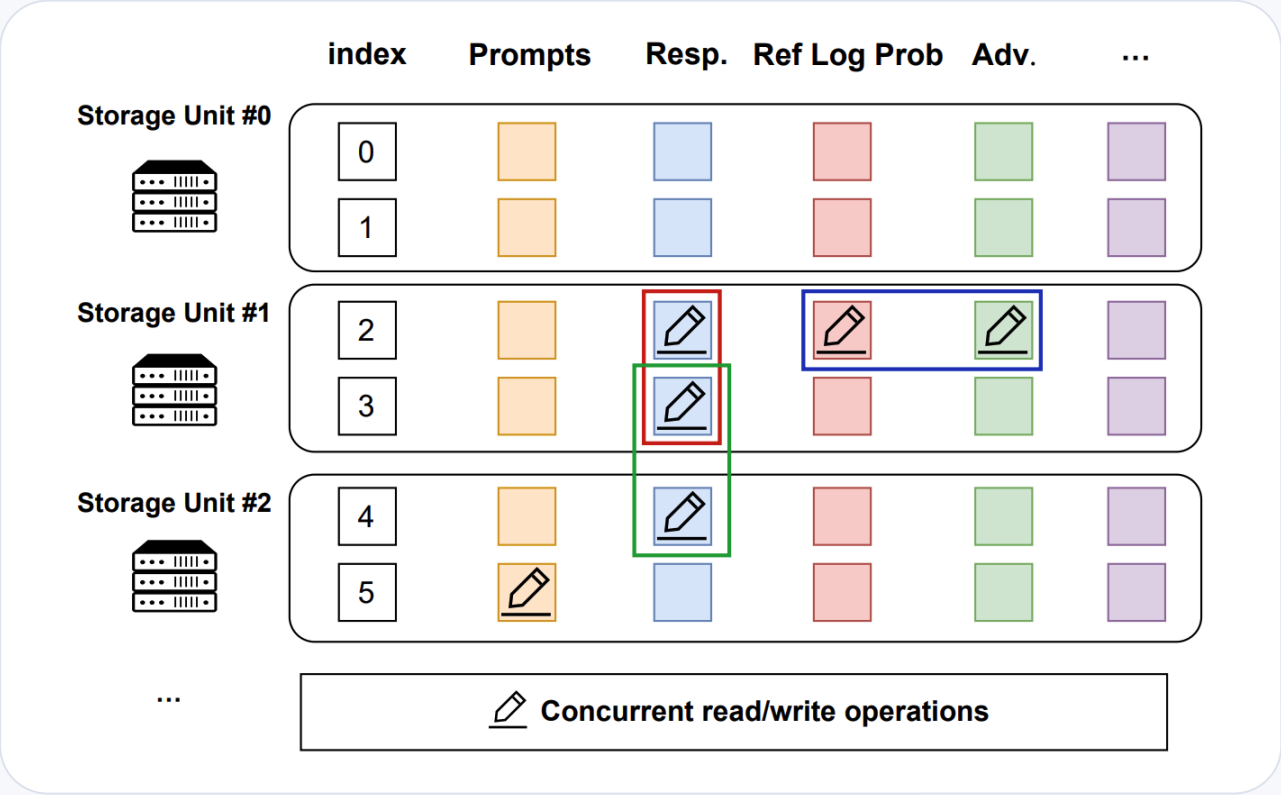
Compose task-specific signals

Setup	Advantage baseline	Learned critic	Framework consequence
PPO + GAE	Value estimates and return targets	Usually enabled	Compute values and update critic
GRPO	Relative reward within a prompt group	Usually disabled	Keep grouped rollout samples together

Reward Loop distributes scoring; streaming scoring is possible for rules or separately resourced reward models.

Algorithms change roles and dependencies; engine and service abstractions remain reusable.

TransferQueue: separate data from control



1 Address fields

Index trajectories and retrieve only the columns a task needs.

2 Track readiness

Metadata records which fields and prompt groups are available.

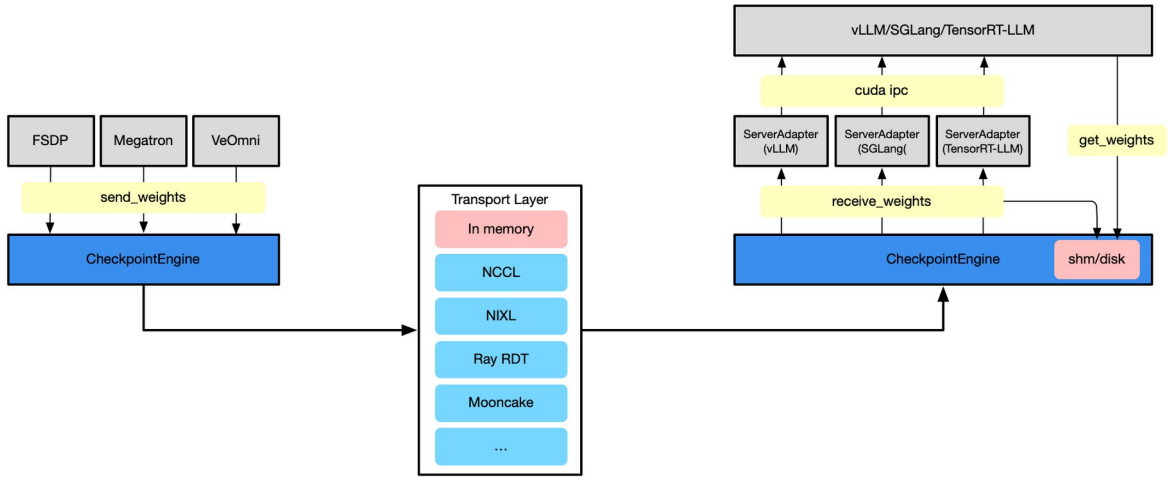
3 Stream consumption

Workers read and write fields; the replay buffer selects trainable groups.

TransferQueue carries sample data. Model updates use the weight-synchronization path.

TransferQueue Data System | Official column-oriented storage diagram; V1 uses TransferQueue

Synchronize policy weights with rollout servers



Live parameter synchronization

Export updated weights and install them in rollout replicas.

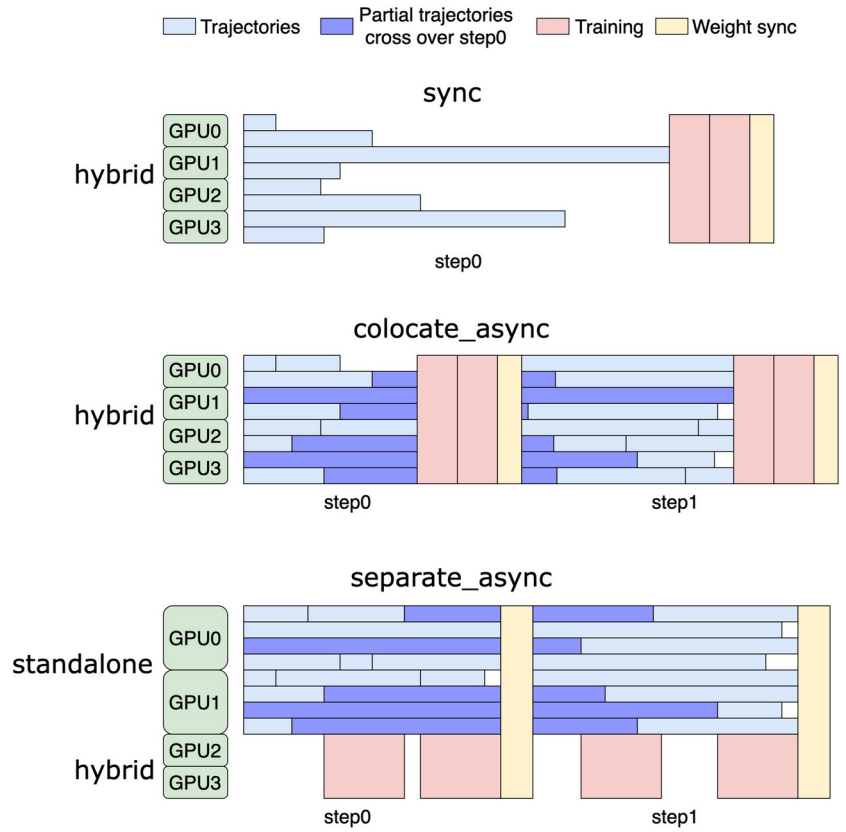
Training checkpoint recovery

Persist model, optimizer, scheduler and trainer state for resume.

Match weight transport to the resource layout and network.

Current checkpoint engine | Official checkpoint-engine figure; current transport registry checked

V1 offers three execution modes



1 **sync**

Finish rollout, train, then synchronize weights. A straightforward baseline.

2 **colocate_async**

Reuse the same pool with warmup batches and partial trajectories across steps.

3 **separate_async**

Run standalone rollout GPUs continuously while training consumes ready mini-batches.

Separate async uses no rollout engine on trainer GPUs by default; optional resource switching is experimental.

V1 Async Trainer | Official current mode timeline

Async training must account for policy versions

ILLUSTRATIVE TRAJECTORY: GENERATED WITH TWO POLICY VERSIONS, TRAINED AT VERSION 7

v5 tokens → **pause** → **v6 tokens** → **train at v7**

- **v5 tokens**: Retained prefix
- **pause**: Mode transition
- **v6 tokens**: Resumed suffix
- **train at v7**: Consume the completed trajectory

Partial rollout

Retain tokens and log probabilities.
Rebuild the KV cache before continuing.
A trajectory can span model versions.

Staleness control

Track version span and sample age.
Current default threshold: 8 versions.
Strategy: drop or wait at prompt-group level.

Throughput gains must be evaluated together with policy mismatch, sample reuse and learning quality.

Configure GRPO: algorithm, sampling and execution

EXAMPLE OVERRIDES

```
algorithm:
  adv_estimator: grpo
actor_rollout_ref:
  actor:
    strategy: fsdp2
  rollout:
    name: vllm
    n: 4
trainer:
  v1:
    trainer_mode: sync
```

1 **Learning signal**

Use group-relative advantage estimation; no default value critic.

2 **Sampling cost**

Generate four responses per prompt; keep each group together.

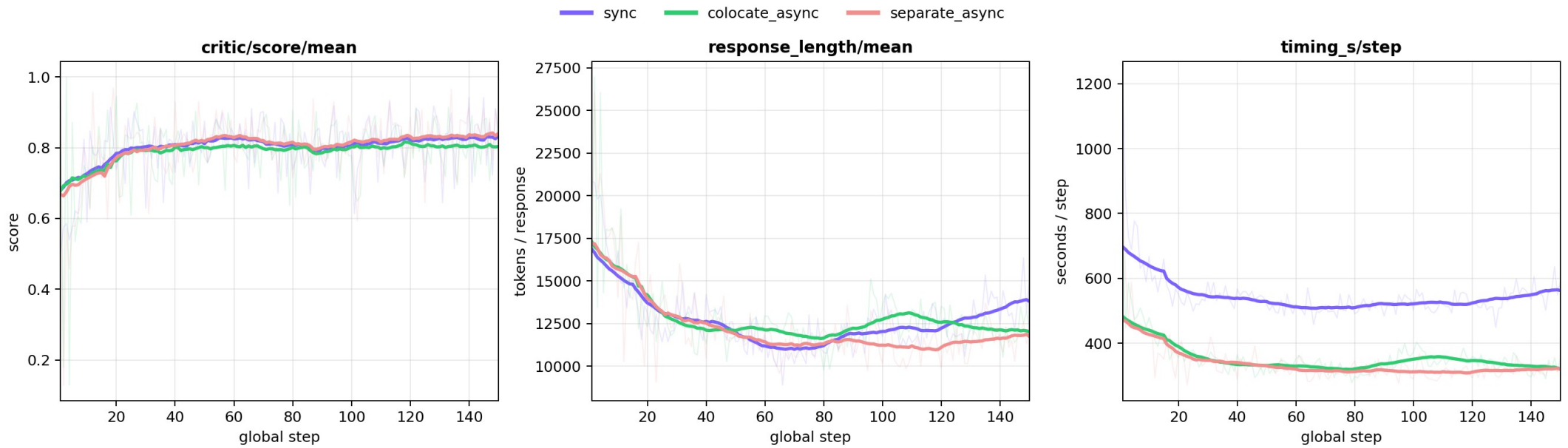
3 **Execution baseline**

Start with synchronized training before comparing async modes.

Add model, dataset, reward and hardware settings using an example that supports the selected engine.

Current PPO configuration | Illustrative YAML overrides, not a complete launch file

Measure learning quality and wall-clock time



Four nodes | Qwen3.5-35B-A3B | Megatron + vLLM | DAPO-Math-17k | max response: 32,768 tokens

sync	colocate_async	separate_async
22.79 h	14.72 h	14.10 h

Also inspect reward, response length, policy staleness, sample-wait time and weight-sync time.

V1 Async Trainer | Reported 150-step comparison in the official V1 guide

Extend verl through focused interfaces

Reward logic

Customize a scoring function or RewardManager. Combine rules, judges and execution feedback.

Agent behavior

Implement AgentLoopBase.run() or provide an external agent-loop manager.

Sampling and scheduling

Customize ReplayBuffer.sample() or V1 lifecycle hooks for filtering and consumption.

Execution backend

Implement and register a BaseEngine, rollout replica or inference-server adapter.

Keep trajectory fields, token alignment and parameter versions consistent across component boundaries.

Key takeaways and official sources

01 Express RL as a dataflow

02 Separate roles from engines

03 Make data and versions explicit

OFFICIAL READING AND CODE MAP

Repository snapshot	Current main branch, pinned to the reviewed commit
V1 trainer guide	Execution modes, partial rollout, staleness and measurements
Engine Workers	Role composition and backend dispatch
Agent Loop	Trajectory structure and multi-turn execution
TransferQueue	Sample data, readiness metadata and streaming
Extension guide	Rewards, agent loops, samplers and engine integrations

Discussion: what would you profile first if rollout dominates the runtime?

RL trainer + model engines + rollout services + trajectory data + weight synchronization.